Being a smart human being (as opposed to a dumb computer program), you can probably answer both questions intuitively: both @x and my_other_method belong to the *receiver*—the object that my_method was originally called upon. However, Ruby doesn't have the luxury of intuition. When you call a method, it needs to tuck away a reference to the receiver. Thanks to this reference, it can remember who the receiver is as it executes the method.

That reference to the receiver can be useful for you as well—so it is worth exploring further.

The self Keyword

Every line of Ruby code is executed inside an object—the so-called *current object*. The current object is also known as self, because you can access it with the self keyword.

Only one object can take the role of self at a given time, but no object holds that role for a long time. In particular, when you call a method, the receiver becomes self. From that moment on, all instance variables are instance variables of self, and all methods called without an explicit receiver are called on self. As soon as your code explicitly calls a method on some other object, that other object becomes self.

Here is an artfully complicated example to show you self in action:

object_model/self.rb

```
class MyClass
  def testing_self
    @var = 10     # An instance variable of self
    my_method()   # Same as self.my_method()
    self
  end

  def my_method
    @var = @var + 1
  end
end

obj = MyClass.new
obj.testing_self  # => #<MyClass:0x007f93ab08a728 @var=11>
```

As soon as you call testing_self, the receiver obj becomes self. Because of that, the instance variable @var is an instance variable of obj, and the method my_method is called on obj. As my_method is executed, obj is still self, so @var is still an instance variable of obj. Finally, testing_self returns a reference to self. (You can also check the output to verify that @var is now 11.)